

Trabalho Prático

Ana Paula Canuto da Silva, 24178
Glória Martins, 22719

Integração de Sistemas de Informação
Prof. Luís Ferreira

Licenciatura em Engenharia em Sistemas Informáticos
(regime pós-laboral)

Escola Superior de Tecnologia
Instituto Politécnico do Cávado e do Ave

Índice

Introdução	2
Trabalho Prático II – Integração de Sistemas de Informação	3
1. Objetivos do Projeto.....	3
2. Tecnologias Utilizadas	3
3. Configuração do Ambiente Cloud	4
3.1. Criação da Conta Azure	4
3.2. Configuração da Base de Dados	5
3.3. Criação da Web App	7
4. Arquitetura da Solução.....	7
4.1. Visão Geral.....	7
4.2. Estrutura de Pastas	8
5. Implementação	9
5.1. Camada Controllers	9
5.2. Camada Data	9
5.3. Pasta Services	10
5.4. Ficheiros de Configuração	11
6. Integração com APIs Externas	12
7. Segurança e Autenticação	12
8. Documentação e Testes.....	13
8.1. Swagger	13
8. 2. Postman.....	14
9. Publicação na Cloud	15
10. Aplicação Cliente iOS	16
11. Repositório GitHub	22
Conclusão e Trabalhos Futuros	23
Referências Bibliográficas.....	25
Apêndice.....	26

Introdução

O presente trabalho foi desenvolvido no âmbito da unidade curricular de Integração de Sistemas de Informação (ISI), do curso de Licenciatura em Engenharia de Sistemas Informáticos do IPCA.

O principal objetivo deste segundo trabalho prático consistiu em desenvolver e explorar serviços web de forma a garantir a interoperabilidade entre sistemas, através da criação de uma API RESTful capaz de realizar operações CRUD e integrar dados provenientes de serviços externos.

O projeto teve como inspiração o primeiro trabalho realizado pela aluna Ana Paula, onde consumia uma API de filmes e tratava os dados extraídos. A aluna Glória abraçou a ideia e propôs a utilização de uma API de livros, povoar uma base de dados em cloud e utilizar esses dados na API RESTful.

A motivação do projeto baseia-se na necessidade crescente de integração e troca de informação entre diferentes aplicações, num contexto em que os sistemas orientados a serviços (SOA) e a computação na cloud assumem um papel central. Desta forma, o trabalho foca-se na implementação de uma solução que permite gerir e relacionar dados de filmes e livros, aproveitando APIs públicas para obter e sincronizar informação real, garantindo assim uma aplicação prática e atual dos conceitos abordados na unidade curricular.

Trabalho Prático II – Integração de Sistemas de Informação

1. Objetivos do Projeto

Os objetivos principais deste trabalho são:

- Desenvolver uma API RESTful completa com operações CRUD sobre entidades de filmes e livros
- Integrar serviços web externos para enriquecer a base de dados
- Implementar um sistema de autenticação seguro
- Publicar a solução na cloud (Microsoft Azure)
- Documentar a API utilizando o standard OpenAPI (Swagger)
- Garantir a interoperabilidade entre sistemas
- Aplicar boas práticas de desenvolvimento e arquitetura de software
- Demonstrar a aplicação prática através de uma aplicação cliente

2. Tecnologias Utilizadas

A solução foi desenvolvida utilizando as seguintes tecnologias:

Backend:

- Linguagem: C#
- Framework: .NET
- ORM: Entity Framework Core

Base de Dados:

- Sistema: SQL Server
- Plataforma Cloud: Microsoft Azure SQL Database

Segurança:

- Encriptação: BCrypt.Net-Next (password hashing)
- Autenticação: JWT (JSON Web Token)

Documentação e Testes:

- Documentação API: Swagger (Swashbuckle.AspNetCore)
- Testes: Postman

APIs Externas:

- TMDb API: The Movie Database (informações sobre filmes)
- Google Books API: Google Books (informações sobre livros)

Cloud:

- Plataforma: Microsoft Azure:
 - Azure SQL Database
 - Azure App Service

3. Configuração do Ambiente Cloud

3.1. Criação da Conta Azure

Para a realização deste trabalho, foi necessário criar uma conta na Microsoft Azure devidamente validada. Inicialmente foi disponibilizado um crédito gratuito para teste que utilizámos na realização deste trabalho:

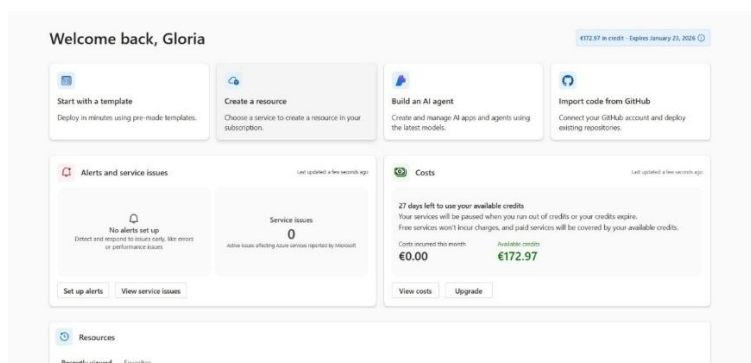


Figura 1 – Criação de conta na Azure para colocar a BD em cloud.

Foi também necessário configurar permissões para que ambas as alunas pudessem trabalhar no mesmo projeto de forma remota e colaborativa:

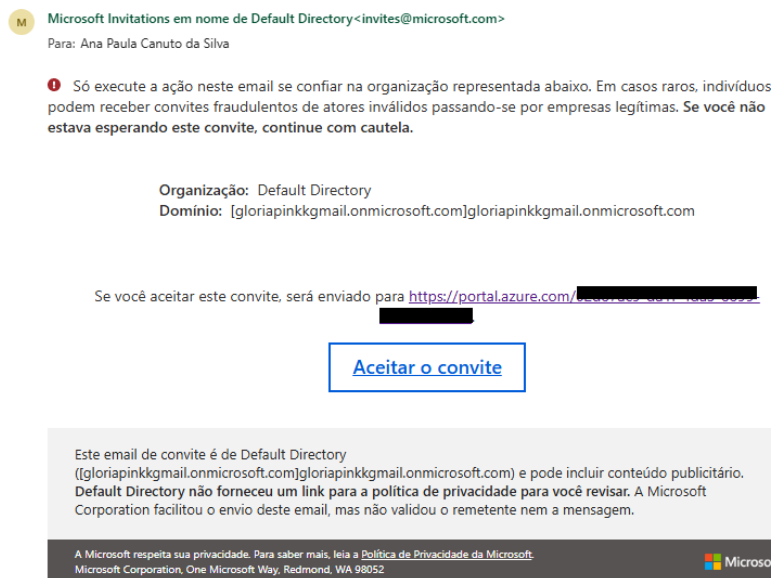


Figura 2 – Permissões para trabalho em equipe de forma remota na plataforma Azure.

3.2. Configuração da Base de Dados

Após a configuração inicial da conta, foi criada a base de dados SQL através da opção "Create Resource" no portal Azure:

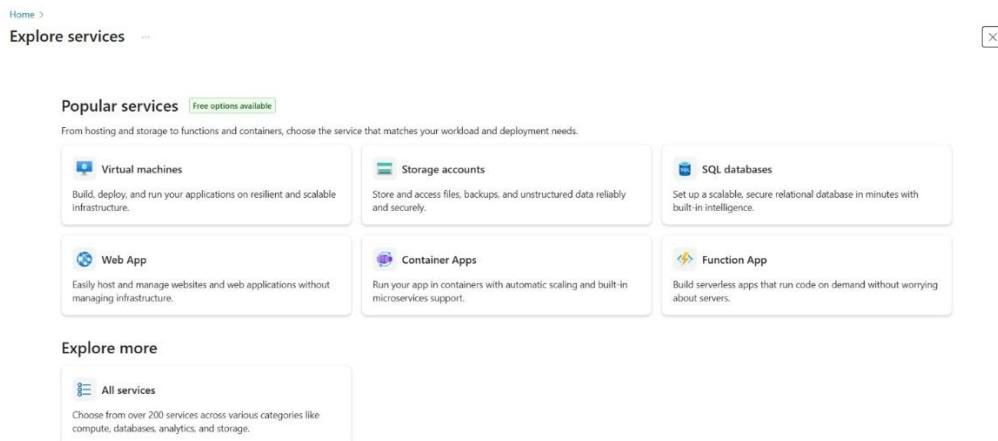


Figura 3 – Dashboard Azure.

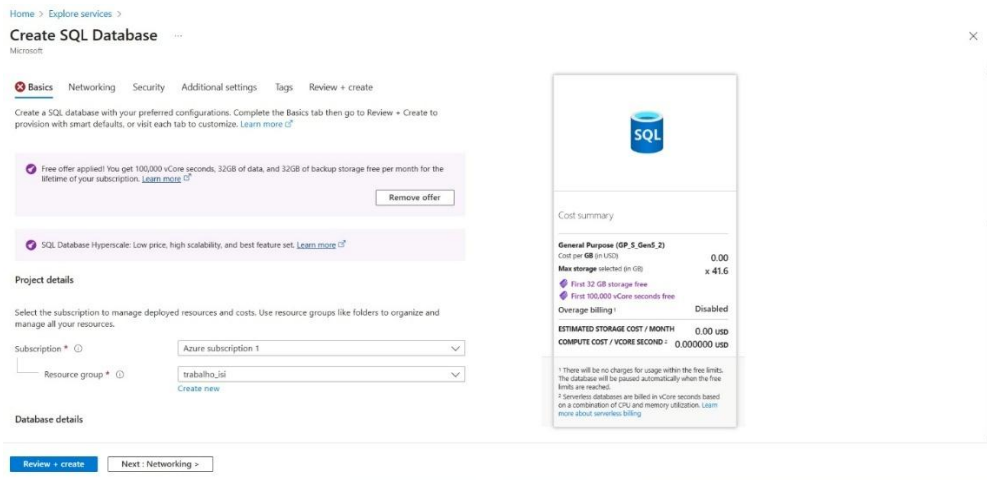


Figura 4 – Criação da Database em SQL.

Foi escolhida a subscrição do tier gratuito, sendo criado um resource group para que da database ficasse associada a ele, além disso foram definidas informações como onde seria alocado o servidor, qual seria o nome da base de dados entre outros.

Script de Criação das Tabelas:

Depois da base de dados criada, as tabelas foram geradas através do seguinte script:

```
CREATE TABLE Movies (
    Id INT PRIMARY KEY,
    Title NVARCHAR(300),
    OriginalTitle NVARCHAR(300),
    Overview NVARCHAR(MAX),
    ReleaseDate NVARCHAR(50),
    PosterPath NVARCHAR(200),
    BackdropPath NVARCHAR(200)
);

CREATE TABLE Books (
    Id NVARCHAR(50) PRIMARY KEY,
    Title NVARCHAR(300),
    Subtitle NVARCHAR(300),
    Authors NVARCHAR(500),
    Publisher NVARCHAR(200),
    PublishedDate NVARCHAR(50),
    Description NVARCHAR(MAX)
);

CREATE TABLE Users (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Username NVARCHAR(100) NOT NULL,
    Email NVARCHAR(256) NOT NULL,
    PasswordHash NVARCHAR(256) NOT NULL,
    Role NVARCHAR(50) DEFAULT 'User',
    CreatedAt DATETIME2 DEFAULT GETUTCDATE()
);
```

3.3. Criação da Web App

Em seguida, foi criada uma Web App na Azure para hospedar a API, onde basicamente só foi necessário fazer o deploy da aplicação:

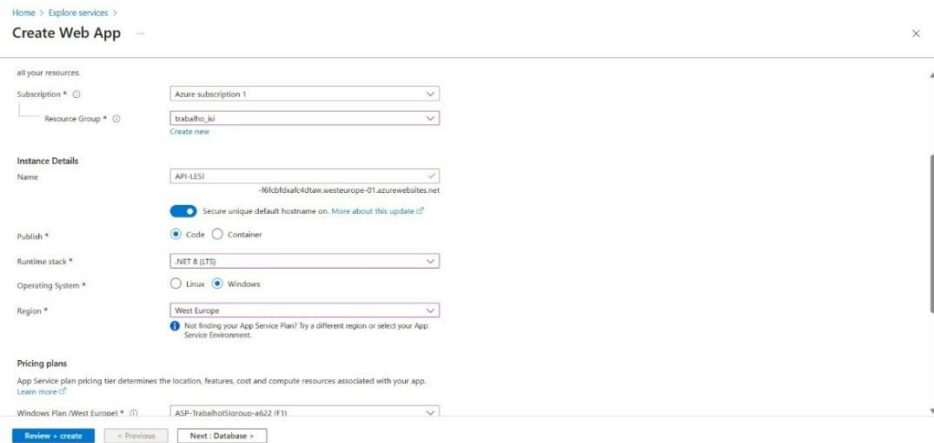


Figura 5 – Criação da Web API na Azure.

Para confirmar que tudo estava a funcionar como esperado e que a base de dados estava a ser povoada corretamente com os dados das APIs externas, foram realizados alguns testes através do Query Editor no portal Azure:

10 itens.

Nome da operação	Estado	Hora	Carimbo de data/hora	Subscrição
> Atualizar base de dados SQL	Com êxito	há 8 minutos	Fri Dec 26 2025 15:57:50 GMT+0000 (...)	Azure subscription 1
> Resume Databases	Com êxito	há 18 minut...	Fri Dec 26 2025 15:47:53 GMT+0000 (...)	Azure subscription 1
> Atualizar base de dados SQL	Com êxito	há uma hora	Fri Dec 26 2025 15:05:24 GMT+0000 (...)	Azure subscription 1
> Pause Databases	Iniciado	há uma hora	Fri Dec 26 2025 14:55:32 GMT+0000 (...)	Azure subscription 1
> Atualizar base de dados SQL	Com êxito	há 2 horas	Fri Dec 26 2025 14:14:22 GMT+0000 (...)	Azure subscription 1
> Resume Databases	Com êxito	há 2 horas	Fri Dec 26 2025 14:04:32 GMT+0000 (...)	Azure subscription 1
> Pause Databases	Iniciado	há 2 horas	Fri Dec 26 2025 14:02:32 GMT+0000 (...)	Azure subscription 1
> Atualizar base de dados SQL	Com êxito	há 2 horas	Fri Dec 26 2025 14:02:27 GMT+0000 (...)	Azure subscription 1
> Atualizar base de dados SQL	Com êxito	há 3 horas	Fri Dec 26 2025 13:23:32 GMT+0000 (...)	Azure subscription 1
> Resume Databases	Com êxito	há 3 horas	Fri Dec 26 2025 13:13:30 GMT+0000 (...)	Azure subscription 1

Figura 6 – Atualização da base de dados em SQL.

4. Arquitetura da Solução

4.1. Visão Geral

A aplicação desenvolvida, denominada "Trabalho_ISI", foi concebida com o objetivo de explorar e consolidar os conceitos de Integração de Sistemas de Informação, focando-se na criação de uma API RESTful capaz de realizar operações CRUD sobre livros e filmes, bem como disponibilizar funcionalidades de autenticação e recomendação.

O projeto foi estruturado de forma organizada e modular, assegurando uma separação clara entre as camadas de dados, serviços e controladores, de acordo com as boas práticas de desenvolvimento de aplicações baseadas em serviços web.

Ter o código devidamente organizado em pastas e de acordo com as suas camadas facilita a atualização e manutenção do código futuramente, caso seja necessário, além de tornar mais fácil a compreensão por parte de outras pessoas que trabalhem no projeto em equipa. Ter o código bem organizado faz parte das boas práticas que devem ser aplicadas na programação, independentemente do projeto ou linguagem utilizada.

4.2. Estrutura de Pastas

O diagrama apresentado representa a estrutura organizacional do projeto, evidenciando a forma como as diferentes camadas da aplicação estão organizadas, mostrando a separação clara entre as responsabilidades do sistema.

No topo do diagrama encontra-se o módulo principal "Trabalho_ISI", que representa a aplicação como um todo. A partir deste ponto, o projeto é dividido em quatro grandes secções principais: Controllers, Data, Services e Properties. Além disso, na base do diagrama são apresentados os ficheiros de configuração e inicialização do sistema.

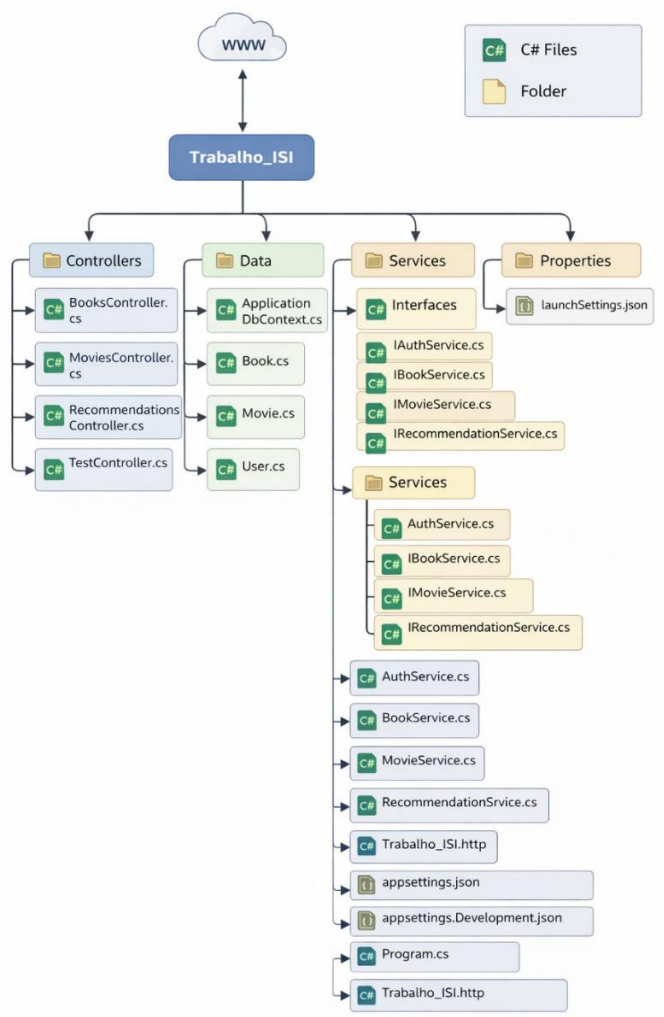


Figura 7 – Diagrama da Estrutura do Projeto.

No topo do diagrama há uma representação de ligação ao WWW, que simboliza a publicação da API na cloud. Esta ligação demonstra que os serviços desenvolvidos podem ser acedidos remotamente, reforçando o conceito de interoperabilidade e de disponibilização de serviços numa arquitetura orientada à cloud (PaaS).

5. Implementação

5.1. Camada Controllers

A camada Controllers contém os controladores responsáveis por expor os serviços RESTful através de endpoints HTTP..

Ficheiros incluídos:

- BooksController.cs
- MoviesController.cs
- RecommendationsController.cs
- TestController.cs

Cada controlador é responsável por gerir um conjunto de operações específicas sobre as entidades correspondentes (livros, filmes e recomendações). Estes ficheiros comunicam com os serviços da aplicação para realizar as operações CRUD e devolver as respostas adequadas às requisições feitas pelos utilizadores ou por aplicações externas.

O TestController foi utilizado para realizar testes e validar o funcionamento da API durante o desenvolvimento.

5.2. Camada Data

A camada Data contém os elementos responsáveis pela gestão da camada de dados da aplicação.

Ficheiros incluídos:

- ApplicationDbContext.cs - Contexto da base de dados utilizando Entity Framework Core
- Book.cs - Entidade de livros
- Movie.cs - Entidade de filmes
- User.cs - Entidade de utilizadores
- DataSeeder.cs - Responsável por inicializar e popular a base de dados com dados das APIs externas

O ficheiro `ApplicationDbContext.cs` representa o contexto da base de dados, utilizando o Entity Framework Core para fazer o mapeamento das entidades e a ligação ao repositório de dados.

As classes `Book`, `Movie` e `User` são as entidades principais do sistema, correspondendo às tabelas existentes na base de dados SQL na Azure.

O ficheiro `DataSeeder.cs` tem como objetivo popular a base de dados com informação proveniente das APIs externas (TMDB API e Google Books API), garantindo dados reais e atualizados para a aplicação.

Esta camada é essencial para garantir a persistência e integridade da informação.

5.3. Pasta Services

A pasta **Services** está dividida em duas partes principais: **Interfaces** e as **implementações** dos serviços.

a) Subpasta Interfaces

Nesta subpasta estão definidos os contratos das funcionalidades disponibilizadas pelos serviços.

Ficheiros incluídos:

- `IAuthService.cs` - Define operações de autenticação
- `IBookService.cs` - Define operações CRUD sobre livros
- `IMovieService.cs` - Define operações CRUD sobre filmes
- `IRecommendationService.cs` - Define lógica de recomendações

Cada interface define um conjunto de métodos que descrevem o comportamento esperado de cada serviço, por exemplo:

- O `IAuthService` define as operações de autenticação, como `RegisterAsync` e `LoginAsync`, responsáveis pelo registo e login de utilizadores.
- O `IBookService` e o `IMovieService` definem as operações CRUD sobre livros e filmes, respetivamente, incluindo pesquisa, paginação e filtragem.
- O `IRecommendationService` implementa a lógica de recomendação cruzada entre livros e filmes, permitindo obter sugestões de acordo com as preferências do utilizador.

b) Implementações dos Serviços

Abaixo das interfaces encontram-se as suas implementações concretas, que dão corpo às funcionalidades definidas anteriormente.

Ficheiros incluídos:

- AuthService.cs
- BookService.cs
- MovieService.cs
- RecommendationService.cs

Cada um destes ficheiros implementa a lógica de negócio associada à sua interface, comunicando diretamente com o ApplicationDbContext da camada de dados.

Esta separação entre interface e implementação facilita a manutenção, promove a reutilização de código e segue o princípio da injeção de dependências, prática comum no desenvolvimento em .NET.

5.4. Ficheiros de Configuração

Pasta Properties:

A pasta Properties contém o ficheiro launchSettings.json, que armazena as definições de execução e ambiente da aplicação, como portas de servidor, perfis de arranque e variáveis de configuração utilizadas durante o desenvolvimento local.

Ficheiros de Configuração na Raiz:

Na parte inferior do diagrama encontram-se os ficheiros responsáveis pela configuração geral do projeto:

- appsettings.json - Configurações principais da aplicação
- appsettings.Development.json - Configurações específicas para ambiente de desenvolvimento
- Program.cs - Responsável pela inicialização da aplicação
- Trabalho_ISI.http - Utilizado para testar requisições HTTP

Os ficheiros appsettings.json e appsettings.Development.json armazenam as configurações da aplicação, como a connection string para a base de dados na Azure e as API keys utilizadas para integrar as APIs externas do IMDB e do Google Books.

O ficheiro Program.cs é responsável pela inicialização da aplicação, configurando os serviços, o acesso à base de dados e o Swagger (OpenAPI), que documenta e permite testar os endpoints de forma interativa.

O ficheiro Trabalho_ISI.http é utilizado para testar diretamente as requisições HTTP, facilitando a validação e o funcionamento das rotas desenvolvidas.

6. Integração com APIs Externas

Para enriquecer os dados da aplicação, foram integradas duas APIs externas:

TMDB API (The Movie Database)

- Utilizada para obter informações sobre filmes
- Dados incluem: título, título original, sinopse, data de lançamento, imagens
- Documentação: <https://developer.themoviedb.org/>

Google Books API

- Utilizada para recolher dados sobre livros
- Dados incluem: título, subtítulo, autores, editora, data de publicação, descrição
- Documentação: <https://developers.google.com/books>

O acesso a ambas as APIs exigiu a utilização de API Keys, que foram configuradas nos ficheiros de definição da aplicação (appsettings.json).

Estas integrações demonstram a capacidade de interoperabilidade entre sistemas distintos, objetivo central da unidade curricular. Os dados recolhidos das APIs foram utilizados para popular a base de dados inicial através do DataSeeder, garantindo dados reais e atualizados.

7. Segurança e Autenticação

A aplicação implementa um sistema de autenticação e segurança que garante o controlo de acesso às operações disponibilizadas pela API.

Componentes de Segurança:

1. Encriptação de Passwords (BCrypt)

Durante o processo de registo, as palavras-passe são encriptadas utilizando a biblioteca **BCrypt.Net-Next**, garantindo que nenhuma senha é armazenada em texto simples na base de dados. Esta abordagem reforça a segurança dos dados sensíveis e segue as boas práticas de proteção de credenciais em sistemas de informação.

2. Autenticação via JWT (JSON Web Token)

O serviço responsável pela autenticação é o `IAuthService`, que gere os processos de registo e login de utilizadores.

Após a autenticação bem-sucedida, o sistema gera um token de acesso no formato **JWT**. Este token contém as informações necessárias para identificar o utilizador e validar a sua sessão, permitindo o controlo de acesso a endpoints protegidos da API.

O token é enviado ao cliente após o login e deve ser incluído em cada requisição subsequente (no header `Authorization` como `Bearer Token`), assegurando que apenas utilizadores autenticados conseguem aceder a determinadas funcionalidades.

Benefícios da Implementação:

Com esta implementação, a aplicação garante um processo de autenticação seguro, fiável e escalável, que poderá ser facilmente expandido futuramente para suportar mecanismos adicionais, como OAuth2 ou autenticação externa através de provedores terceiros.

8. Documentação e Testes

8.1. Swagger

A documentação da API foi gerada com recurso ao Swagger, integrado no projeto através do pacote `Swashbuckle.AspNetCore`, instalado via NuGet.

Esta ferramenta permitiu criar automaticamente a documentação da API de acordo com o standard OpenAPI, apresentando de forma clara todos os endpoints disponíveis, os parâmetros utilizados, os tipos de retorno e os códigos de resposta HTTP.

Configuração do Swagger:

A configuração foi realizada no ficheiro `Program.cs`, incluindo as instruções:

```
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();
```

E no pipeline de execução:

```
app.UseSwagger();  
app.UseSwaggerUI();
```

Para garantir que o Swagger permanecesse disponível mesmo após a publicação na Microsoft Azure, foi configurado para ser sempre exibido, independentemente do ambiente (desenvolvimento ou produção).

Adicionalmente, foram ativados os comentários XML no projeto, permitindo que as descrições dos controladores e métodos fossem automaticamente exibidas no Swagger, tornando a documentação mais completa e legível.

Integração com JWT:

Como a aplicação utiliza JWT para autenticação, o Swagger foi configurado para incluir um esquema de segurança "**Bearer**", possibilitando inserir o token de acesso através do botão "Authorize" e testar endpoints protegidos diretamente na interface de documentação.

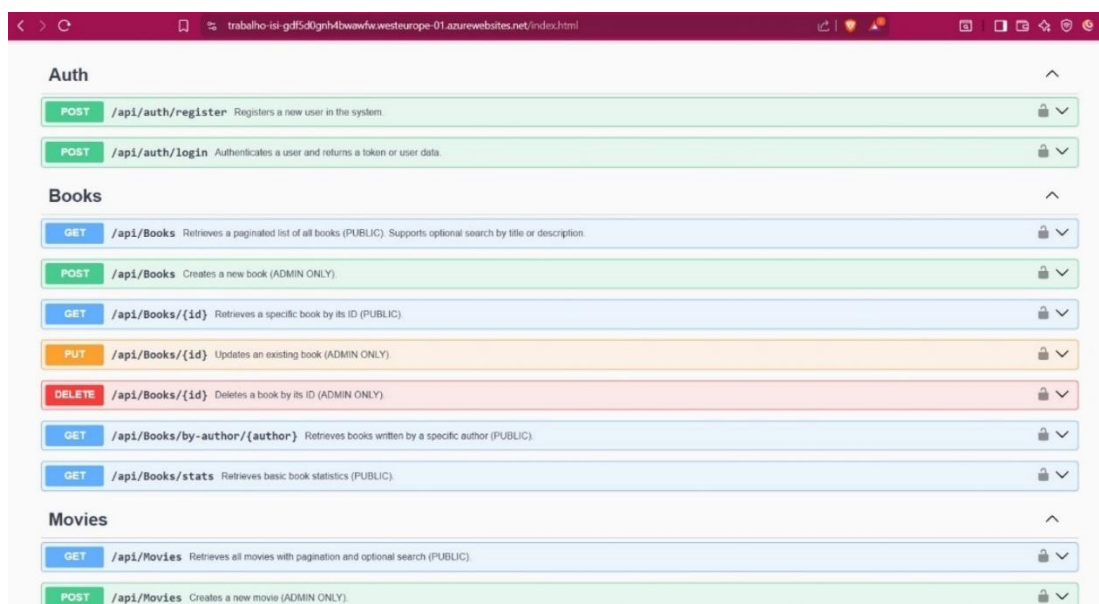


Figura 8 – Utilização do Swagger para documentação da API.

8. 2. Postman

Para os testes funcionais da API, foi utilizada a ferramenta **Postman**, que permitiu validar o comportamento de todos os endpoints RESTful.

Testes Realizados:

- Operações CRUD de livros e filmes
- Autenticação de utilizadores (registo e login)
- Funcionalidade de recomendações
- Requisições e respostas HTTP
- Códigos de estado (200, 201, 400, 404, 500, etc.)

- Tratamento de erros
- Coerência dos dados retornados

Através do Postman, foi possível verificar o correto funcionamento de cada endpoint, garantindo que a API responde adequadamente a diferentes cenários de uso.

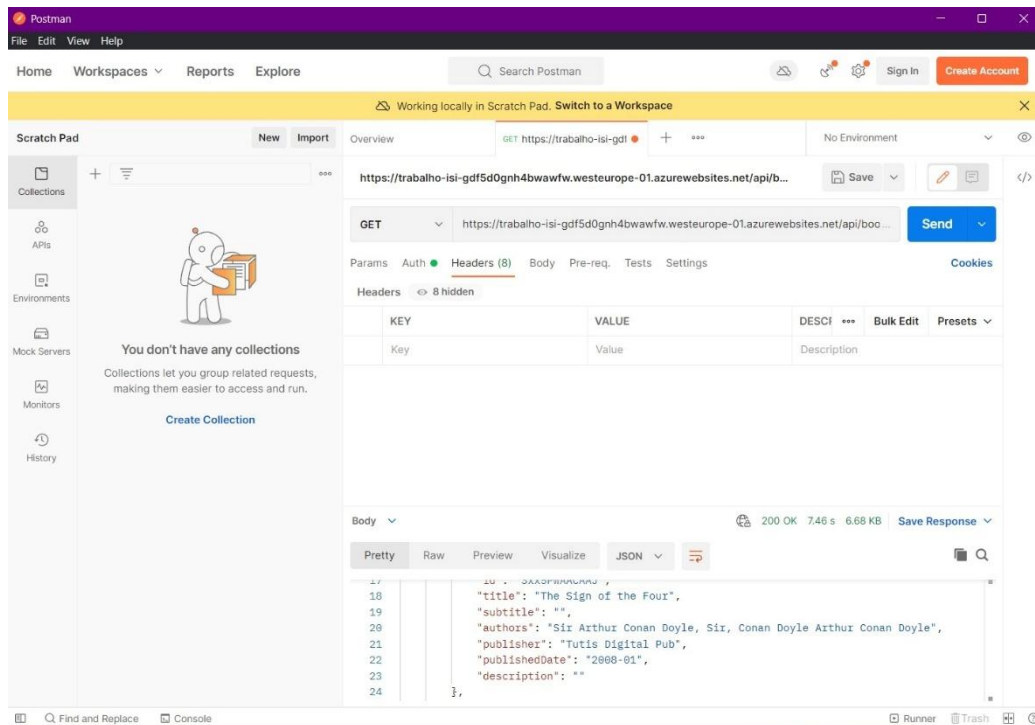


Figura 9 – Utilizando o Postman para realização dos testes.

Desta forma, o **Swagger** garantiu uma documentação interativa e completa da API, enquanto o **Postman** proporcionou uma validação prática e detalhada dos serviços implementados, assegurando o correto funcionamento e a conformidade da solução com os requisitos definidos.

9. Publicação na Cloud

O projeto foi configurado para ser publicado na **Microsoft Azure**, incluindo tanto a base de dados como a API RESTful.

Componentes Publicados:

1. Azure SQL Database

- Base de dados relacional hospedada na Azure

- Tier gratuito utilizado para o desenvolvimento
- Tabelas: Movies, Books, Users
- Dados populados através do DataSeeder com informação das APIs externas

2. Azure App Service (Web API)

- Aplicação ASP.NET Core hospedada na Azure
- Runtime: .NET
- Endpoints RESTful acessíveis publicamente
- Swagger disponível online para documentação interativa

Processo de Publicação:

A publicação foi realizada através do Visual Studio, utilizando a funcionalidade de deploy integrada para Azure App Services. As configurações de connection string e API keys foram definidas diretamente no portal Azure, garantindo a segurança das credenciais.

Esta publicação assegura que os serviços desenvolvidos estão acessíveis online, reforçando o carácter distribuído, interoperável e escalável da solução proposta, cumprindo os requisitos de publicação em ambiente cloud definidos no enunciado da unidade curricular.

10. Aplicação Cliente iOS

Após a conclusão do projeto, com a database e API RESTful em cloud na Azure, documentação no Swagger e testes no Postman, foi desenvolvida uma **aplicação iOS** para consumir a API.

Objetivo:

A aplicação foi criada para demonstrar uma possível utilização prática da solução desenvolvida, permitindo aos utilizadores interagir com os dados de filmes e livros através de uma interface móvel intuitiva e com design clean.

Funcionalidades Implementadas:

A aplicação permite:

- Pesquisar livros e filmes
- Editar informações
- Adicionar novos registos
- Eliminar registos existentes

Um caso de uso interessante é a possibilidade de, ao saber o nome de um filme, fazer a busca entre os livros e descobrir se o filme foi inspirado em algum livro com o mesmo nome, ou o inverso.

Estrutura da Aplicação:

A aplicação está dividida em quatro secções principais:

1. **Books** - Gestão de livros
2. **Movies** - Gestão de filmes
3. **Recommendations** - Recomendações cruzadas
4. **Logout** - Terminar sessão

Capturas de Ecrã

Secção Books:

Na imagem seguinte é possível ver a aba onde mostra a lista dos livros, com o campo para busca no topo e na parte inferior a opção de trocar para as outras abas.

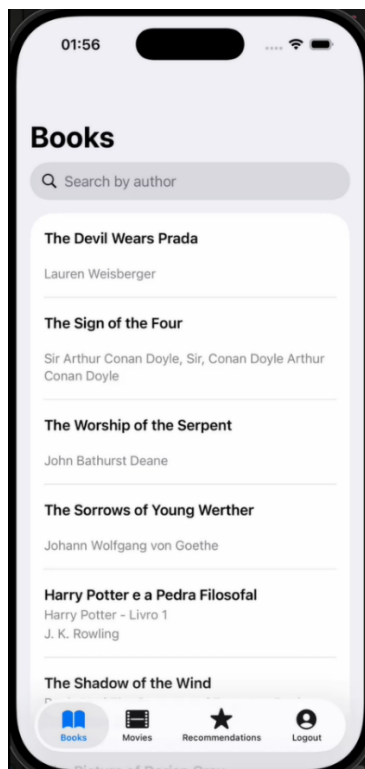


Figura 10 – Aplicação iOS na aba Books.

A ferramenta de busca funciona para encontrar livros específicos, incluindo pesquisa por autor. Na imagem seguinte, foi realizada uma pesquisa pela autora Lauren Weisberger.

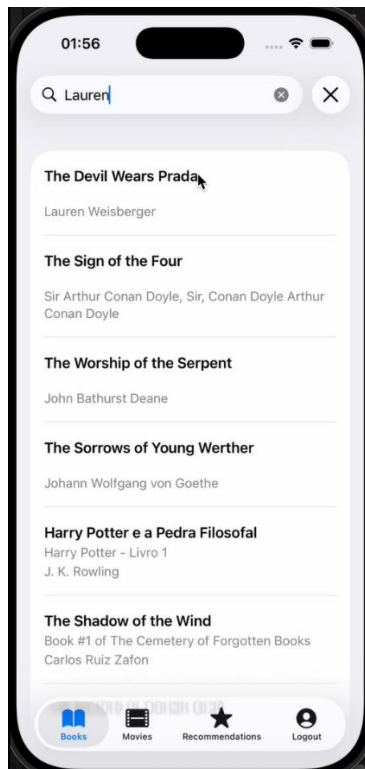


Figura 11 – Utilização da ferramenta de busca.

Secção Movies:

Na aba dos filmes, intitulada "Movies", muito similar à aba anterior, é possível verificar a lista de filmes. Para testes, foi adicionado o filme "Glória" utilizando os serviços da API.



Figura 12 – Aplicação iOS na aba Movies.

Funcionalidade de Edição:

A aplicação permite editar informações dos registos. Na imagem seguinte, vemos as telas de edição onde é possível modificar ou excluir as informações de um filme. Neste exemplo, foi adicionada a palavra "Editada" ao título do filme.

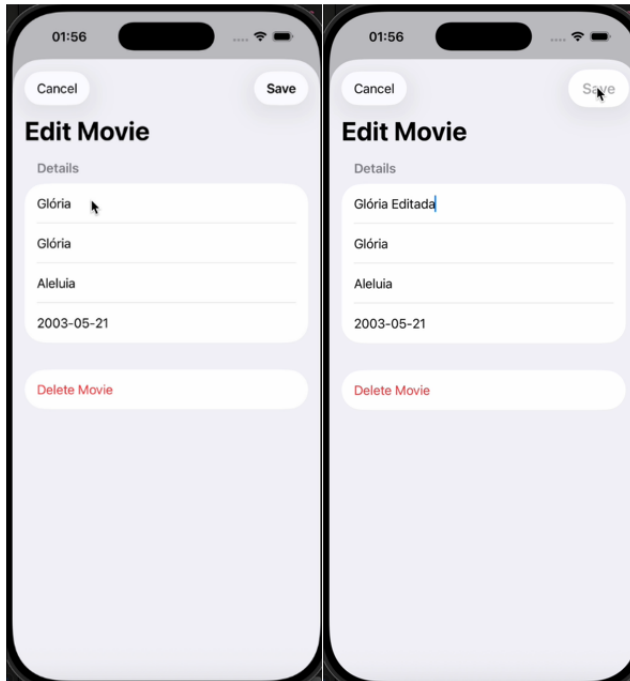


Figura 13 – Edição de informações utilizando a app.

Ao retornar à lista, é possível verificar que o título do filme já aparece modificado.



Figura 14 – Lista mostra modificação realizada no nome do filme.

Nota: Em uma situação real, estas funcionalidades de edição serviriam para corrigir possíveis erros. Obviamente, numa aplicação real, tais permissões poderiam ficar limitadas a administradores, ou serem utilizadas para criar listas de livros/filmes favoritos do utilizador.

Recommendations:

Recomendações cruzadas, essa funcionalidade compara automaticamente a lista de filmes com a lista de livros, identificando correspondências através dos títulos. Quando existe um livro associado a um determinado filme (ou vice-versa), a aplicação apresenta essa informação de forma clara ao utilizador.

Vantagens desta Funcionalidade:

- Descoberta de Adaptações Cinematográficas
- O utilizador pode descobrir que o seu livro favorito teve uma adaptação para cinema, ou que determinado filme foi inspirado numa obra literária.
- Imersão na História
- Permite ao utilizador explorar a mesma narrativa em diferentes formatos, proporcionando uma experiência cultural mais rica e completa.
- Continuidade de Sagas Literárias
- Em séries literárias onde nem todos os livros foram adaptados ao cinema, o utilizador pode descobrir os livros seguintes da saga, garantindo a continuidade da história para além dos filmes disponíveis.
- Enriquecimento Cultural
- Promove o cruzamento entre literatura e cinema, incentivando o consumo de ambos os formatos.

Exemplo Prático:

Na imagem abaixo, é possível observar um exemplo desta funcionalidade em ação, onde a aplicação identifica que existe tanto um filme como um livro da obra "Harry Potter e a Pedra Filosofal". Esta correspondência permite ao utilizador explorar a história tanto através da leitura como do visionamento, ou ainda descobrir os livros seguintes da saga caso pretenda continuar a aventura para além do primeiro filme.

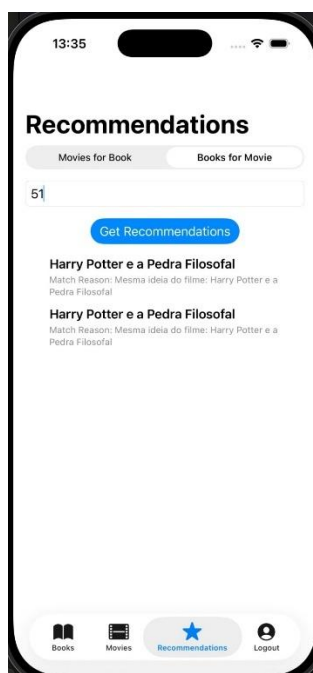


Figura 15 - Funcionalidade de recomendações cruzadas identificando correspondência entre filme e livro.

Esta funcionalidade demonstra o potencial de integração de dados provenientes de diferentes fontes (TMDB API e Google Books API) e a criação de valor através do cruzamento inteligente de informação, um dos objetivos centrais da Integração de Sistemas de Informação.

Logout:

A aplicação inclui também a funcionalidade de logout, permitindo ao utilizador terminar a sessão de forma segura.

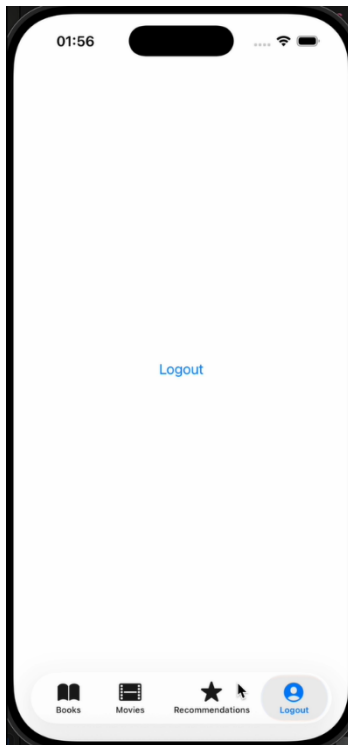


Figura 16 -Tela de Logout na aplicação iOS.

11. Repositório GitHub

Repositório:

Para além da documentação presente neste relatório, o código fonte completo do projeto foi disponibilizado num repositório público no **GitHub**, permitindo o acesso e consulta de toda a implementação desenvolvida.

URL do Repositório:

<https://github.com/XIceQueenX/ProjetoISI>

O repositório encontra-se devidamente organizado e inclui um ficheiro **README.md** que fornece informações essenciais sobre o projeto, nomeadamente:

- **Descrição geral** da API e suas funcionalidades
- **Tecnologias utilizadas** no desenvolvimento
- **Estrutura do projeto** com explicação das principais pastas e ficheiros
- **Instruções de configuração** para execução local da aplicação
- **Endpoints disponíveis** na API RESTful
- **Informações de autenticação** e segurança implementadas
- **Exemplos de utilização** dos serviços desenvolvidos

A disponibilização do código no GitHub demonstra transparência no processo de desenvolvimento e facilita a colaboração, revisão de código e eventual reutilização da solução por outros desenvolvedores. Adicionalmente, o controlo de versões através do Git permitiu uma gestão eficiente do trabalho colaborativo entre as duas alunas ao longo do desenvolvimento do projeto.

O README está formatado de forma clara e profissional, servindo como documentação técnica complementar a este relatório académico, focando-se nos aspetos práticos de implementação e utilização da API.

Conclusão e Trabalhos Futuros

O desenvolvimento deste trabalho permitiu consolidar conhecimentos sobre integração de sistemas, arquiteturas orientadas a serviços e desenvolvimento de APIs RESTful.

A implementação dos serviços de autenticação, gestão de livros e filmes, e recomendações demonstrou a aplicabilidade prática dos conceitos teóricos da unidade curricular, aliando interoperabilidade, integração de serviços externos e publicação em ambiente cloud.

Principais Realizações:

- Desenvolvimento de uma API RESTful completa com operações CRUD
- Integração com APIs externas (TMDb e Google Books) para enriquecimento de dados
- Implementação de sistema de autenticação seguro com JWT e BCrypt
- Publicação da solução na Microsoft Azure (base de dados e Web API)
- Documentação automática da API com Swagger (OpenAPI)
- Testes funcionais utilizando Postman
- Desenvolvimento de aplicação cliente iOS para demonstração prática

O uso de APIs externas enriqueceu o repositório de dados e evidenciou a importância da reutilização de informação existente. A estrutura modular do código e a documentação com Swagger facilitam a manutenção e futura expansão do sistema, permitindo a integração com outras aplicações ou microserviços.

Trabalhos Futuros:

Caso seja decidido dar continuidade a este projeto, as seguintes melhorias poderiam ser implementadas:

- Criação de serviços adicionais na Web API
- Desenvolvimento de aplicação Android
- Criação de um website onde utilizadores pudessem:
 - Criar contas personalizadas
 - Verificar filmes e livros mais consumidos atualmente
 - Pesquisar quais filmes foram baseados em livros
 - Aceder a estatísticas e curiosidades
- Integração com plataformas de venda de livros ou entradas para cinema

- Implementação de sistema de recomendações mais avançado baseado em machine learning
- Expansão das funcionalidades de pesquisa e filtragem

A Web API desenvolvida tem potencial para ser utilizada em diversos contextos, desde plataformas educacionais até sistemas comerciais de venda de conteúdos culturais.

.

Referências Bibliográficas

- Microsoft .NET Documentation – Official Microsoft Learn. Disponível em: <https://learn.microsoft.com/dotnet/>.
- C# Language Reference – Microsoft Learn. Disponível em: <https://learn.microsoft.com/dotnet/csharp/>
- Entity Framework Core – Official Documentation. Disponível em: <https://learn.microsoft.com/ef/core/>
- Swagger (OpenAPI) – Official Documentation. Disponível em: <https://swagger.io/docs/specification/about>
- Microsoft Azure – Cloud Platform Documentation. Disponível em: <https://learn.microsoft.com/azure>
- BCrypt.Net-Next – Official GitHub Repository. Disponível em: <https://github.com/BcryptNet/bcrypt.net>
- JWT.io – JSON Web Tokens Introduction and Documentation. Disponível em: <https://jwt.io/introduction>
- TMDb API Documentation – *The Movie Database Developers*. Disponível em: <https://developer.themoviedb.org/>
- Google Books API – Google Developers. Disponível em: <https://developers.google.com/books>
- Stack Overflow – Discussões sobre .NET, Entity Framework e JWT Authentication. Disponível em: <https://stackoverflow.com/>

Apêndice

Figura	Descrição
Figura 1	Criação de conta na Azure para colocar a BD em cloud.
Figura 2	Permissões para trabalho em equipa de forma remota na plataforma Azure
Figura 3	Dashboard Azure.
Figura 4	Criação da Database em SQL.
Figura 5	Criação da Web API na Azure.
Figura 6	Atualização da base de dados em SQL..
Figura 7	Diagrama da Estrutura do Projeto.
Figura 8	Utilização do Swagger para documentação da API.
Figura 9	Utilizando o Postman para realização dos testes.
Figura 10	Aplicação iOS na aba Books
Figura 11	Utilização da ferramenta de busca.
Figura 12	Aplicação iOS na aba Movies.
Figura 13	Edição de informações utilizando a app.
Figura 14	Lista mostra modificação realizada no nome do filme.
Figura 15	Funcionalidade de recomendações cruzadas identificando correspondência entre filme e livro.
Figura 16	Tela de Logout na aplicação iOS.